

Anexa B

Codul Sursa Hardware (VHDL) - Interfata UART

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3 use IEEE.NUMERIC_STD.ALL;
4 use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6 entity uart_axi_interface is
7   Port (
8     clk : in STD_LOGIC;
9     rx_pin : in STD_LOGIC;
10    tx_pin : out STD_LOGIC;
11    gpio_tx_data : in STD_LOGIC_VECTOR(7 downto 0);
12    gpio_tx_start : in STD_LOGIC;
13    gpio_rx_en : in STD_LOGIC;
14    gpio_rx_data : out STD_LOGIC_VECTOR(7 downto 0);
15    gpio_rx_ready : out STD_LOGIC;
16    gpio_tx_rdy : out STD_LOGIC
17  );
18 end uart_axi_interface;
19
20 architecture Behavioral of uart_axi_interface is
21
22   component transmitfsm
23   Port ( clk: in STD_LOGIC;
24         rst : in STD_LOGIC;
25         baud_en : in STD_LOGIC;
26         tx_en : in STD_LOGIC;
27         tx_data : in STD_LOGIC_VECTOR (7 downto 0);
28         tx : out STD_LOGIC;
29         tx_rdy : out STD_LOGIC);
30   end component;
31
32   component receivefsm
33   Port ( clk : in STD_LOGIC;
34         rst : in STD_LOGIC;
35         baud_en : in STD_LOGIC;
36         rx : in STD_LOGIC;
37         rx_data : out STD_LOGIC_VECTOR (7 downto 0);
38         rx_rdy : out STD_LOGIC);
39   end component;
40
41
42   signal baud_en, baud_en_x16 : STD_LOGIC;
43   signal cnt : STD_LOGIC_VECTOR(13 downto 0) := (others => '0');
44   signal cnt_x16 : STD_LOGIC_VECTOR(9 downto 0) := (others => '0');
45
46   --A circular buffer for recieving data - this fixes the garbage values we get
47   constant FIFO_DEPTH : integer := 64;
48   type fifo_mem_type is array (0 to FIFO_DEPTH-1) of std_logic_vector(7 downto 0);
49   signal fifo_mem : fifo_mem_type := (others => (others => '0'));
50
51   signal fifo_wr_ptr : integer range 0 to FIFO_DEPTH-1 := 0;
52   signal fifo_rd_ptr : integer range 0 to FIFO_DEPTH-1 := 0;
53   signal fifo_count : integer range 0 to FIFO_DEPTH := 0;
54
55   signal rx_byte : std_logic_vector(7 downto 0);
56   signal rx_ready : std_logic;
57
58   signal rx_en_q : std_logic;
59   signal rx_ready_q : std_logic := '0';
60   --stores the previous state of rx_ready, because rx_ready stays high for 8 clock
   cycles
61
62   --RX WRITE EN is toggled when the recieved byte is ready for reading.
63   --DO NOT MISTAKE IT FOR RX_EN which is a signal from the CPU to read from the buffer!
64   signal rx_write_en : std_logic;
```

```

65     signal pop_request : std_logic;
66 begin
67
68     edge_detector_rx_ready: process(clk)
69     begin
70         if rising_edge(clk) then
71             rx_ready_q <= rx_ready;
72         end if;
73     end process;
74
75     rx_write_en <= rx_ready and not rx_ready_q;
76
77     edge_detector_pop_request: process(clk)
78     begin
79         if rising_edge(clk) then
80             rx_en_q <= gpio_rx_en;
81         end if;
82     end process;
83     --only pop one element on the rising edge
84     pop_request <= gpio_rx_en and not rx_en_q;
85
86     fifo_controller: process(clk)
87     begin
88         if rising_edge(clk) then
89
90             -- CASE A: Read and Write happen at the same time
91             if (rx_write_en = '1' and fifo_count < FIFO_DEPTH) and (pop_request = '1' and
92                 fifo_count > 0) then
93                 fifo_mem(fifo_wr_ptr) <= rx_byte;
94                 fifo_wr_ptr <= (fifo_wr_ptr + 1) mod FIFO_DEPTH;
95                 fifo_rd_ptr <= (fifo_rd_ptr + 1) mod FIFO_DEPTH;
96
97             -- CASE B: Fifo Write
98             elsif rx_write_en = '1' and fifo_count < FIFO_DEPTH then
99                 fifo_mem(fifo_wr_ptr) <= rx_byte;
100                 fifo_wr_ptr <= (fifo_wr_ptr + 1) mod FIFO_DEPTH;
101                 fifo_count <= fifo_count + 1;
102
103             -- CASE C: Fifo Read
104             elsif pop_request = '1' and fifo_count > 0 then
105                 fifo_rd_ptr <= (fifo_rd_ptr + 1) mod FIFO_DEPTH;
106                 fifo_count <= fifo_count - 1;
107             end if;
108         end if;
109     end process;
110
111     gpio_rx_ready <= '1' when fifo_count > 0 else '0';
112     gpio_rx_data <= fifo_mem(fifo_rd_ptr) when fifo_count > 0 else (others => '0');
113
114     -- 100,000,000 / 9600 = 10416 - 1 = 10415
115     process(clk)
116     begin
117         if rising_edge(clk) then
118             if cnt = 10415 then
119                 baud_en <= '1';
120                 cnt <= (others => '0');
121             else
122                 baud_en <= '0';
123                 cnt <= cnt + 1;
124             end if;
125         end if;
126     end process;
127
128     process(clk)
129     begin
130         if rising_edge(clk) then
131             if cnt_x16 = 650 then -- 10415 / 16
132                 baud_en_x16 <= '1';
133                 cnt_x16 <= (others => '0');
134             else
135                 baud_en_x16 <= '0';
136                 cnt_x16 <= cnt_x16 + 1;
137             end if;

```

```

137     end if;
138 end process;
139
140 inst_TFSM: transmitfsm port map(
141     clk => clk,
142     rst => '0',
143     baud_en => baud_en,
144     tx_en => gpio_tx_start,
145     tx_data => gpio_tx_data,
146     tx => tx_pin,
147     tx_rdy => gpio_tx_rdy
148 );
149
150 inst_RFSM: receivefsm port map(
151     clk => clk,
152     rst => '0',
153     baud_en => baud_en_x16,
154     rx => rx_pin,
155     rx_data => rx_byte,
156     rx_rdy => rx_ready
157 );
158
159 end Behavioral;

```

Listing 1: Implementarea modulului uart_axi_interface